

Honeypot-TI-LLM-Splunk Write-Up

Tahmid Abtahee · Threat Intelligence & Automated Incident Response Lab · July 2026

Why I Built This

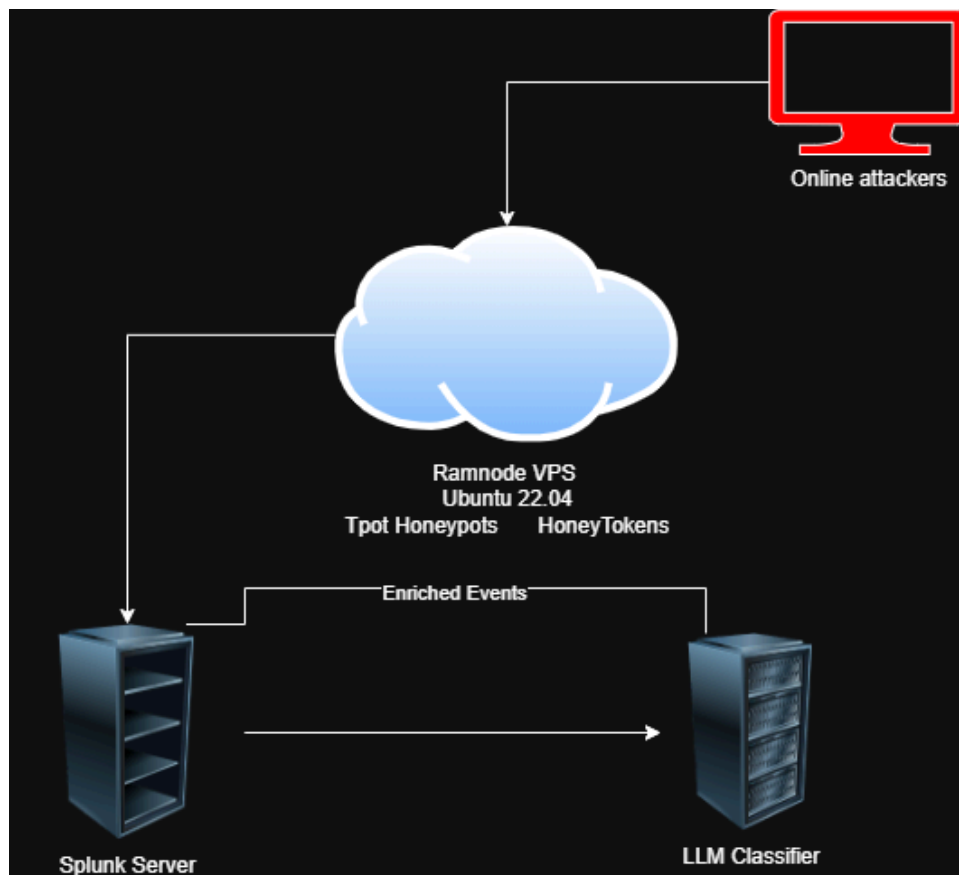
This is the second of two portfolio pipeline projects, following an Active Directory SOC lab and a hybrid detection engineering lab on AWS, Terraform, and Splunk. This one moves from simulated telemetry to real attacker behavior: a live, internet-facing honeypot instead of a closed lab network.

The scope was four concrete goals: stand up a real, internet-exposed honeypot suite instead of simulated traffic; centralize that telemetry in Splunk at a scale beyond a single log source; add a deception layer with honeytokens to go beyond passive log collection; and build an automated pipeline where an LLM classifies raw events against MITRE ATT&CK and writes the enrichment back into Splunk, effectively a lightweight, automated tier-1 triage step.

Within the first 24 hours of going live, the honeypot logged real attack traffic from the US, Seychelles, Portugal, South Korea, China, and more. By the time this was written, over 1,100 events were indexed, the LLM pipeline was validated end to end against live data, and two honeytokens were deployed and confirmed working.

Architecture

Internet attackers reach the exposed honeypot services on a single VPS. T-Pot's containers and the honeypot deception layer both run there. Honeypot logs are ingested into Splunk, which acts as the central SIEM. A Python classifier periodically pulls raw events from Splunk, sends them to Claude for MITRE ATT&CK classification, and writes the enriched result back into Splunk under its own sourcetype.



End-to-end architecture: attackers to T-Pot honeypots and honeytokens on the VPS, to Splunk, to the LLM classifier, and back into Splunk as enriched events.

Running Splunk and T-Pot on the same box was a deliberate tradeoff: it avoided the networking and tunneling complexity of a second machine, at the cost of resource contention between the two services, covered below.

Infrastructure & Honeypot Deployment

What I Did

The lab runs on a RamNode KVM VPS (Ubuntu 22.04, EWR region), originally sized at 8GB RAM / 4 vCPU to keep initial cost low. T-Pot 24.04.1 was installed in Hive mode, deploying 20+ honeypot daemons, including Cowrie (fake SSH/Telnet), Dionaea (malware and exploit capture), Honeytrap (broad catch-all listener), Conpot (ICS protocol simulation), and Suricata (network IDS). Each honeypot runs in its own Docker container, with docker-proxy forwarding host ports into the corresponding container.

```
Active: active (running) since Sun 2026-07-12 23:44:48 UTC; 4min 40s ago
TriggeredBy: ● docker.socket
Docs: https://docs.docker.com
Main PID: 776 (dockerd)
Tasks: 958
Memory: 352.1M
CPU: 19.088s
CGroup: /system.slice/docker.service
└─ 776 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
    3084 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 9200 -container-ip 172.19.0.2 -container-port 9200
    3101 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 9200 -container-ip 172.19.0.2 -container-port 9200
    3186 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 3389 -container-ip 192.168.144.2 -container-port 3389
    3205 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 3389 -container-ip 192.168.144.2 -container-port 3389
    3349 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 9100 -container-ip 192.168.80.2 -container-port 9100
    3407 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 9100 -container-ip 192.168.80.2 -container-port 9100
    3606 /usr/bin/docker-proxy -proto tcp -host-ip 127.0.0.1 -host-port 64288 -container-ip 172.18.0.2 -container-port 64288
    3813 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8080 -container-ip 172.24.0.2 -container-port 8080
    3845 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8080 -container-ip 172.24.0.2 -container-port 8080
    4040 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 10001 -container-ip 172.23.0.2 -container-port 10001
    4108 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 10001 -container-ip 172.23.0.2 -container-port 10001
    4305 /usr/bin/docker-proxy -proto udp -host-ip 0.0.0.0 -host-port 5000 -container-ip 192.168.0.2 -container-port 5000
    4333 /usr/bin/docker-proxy -proto udp -host-ip :: -host-port 5000 -container-ip 192.168.0.2 -container-port 5000
    4409 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8443 -container-ip 192.168.0.2 -container-port 8443
    4418 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8443 -container-ip 192.168.0.2 -container-port 8443
    4475 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 5060 -container-ip 192.168.64.2 -container-port 5060
    4482 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 5060 -container-ip 192.168.64.2 -container-port 5060
    4519 /usr/bin/docker-proxy -proto udp -host-ip 0.0.0.0 -host-port 5060 -container-ip 192.168.64.2 -container-port 5060
    4527 /usr/bin/docker-proxy -proto udp -host-ip :: -host-port 5060 -container-ip 192.168.64.2 -container-port 5060
    4580 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 25 -container-ip 172.21.0.2 -container-port 25
    4589 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 25 -container-ip 172.21.0.2 -container-port 25
    4614 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 587 -container-ip 172.21.0.2 -container-port 587
    4619 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 587 -container-ip 172.21.0.2 -container-port 587
    4648 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 64294 -container-ip 172.18.0.4 -container-port 64294
    4653 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 64294 -container-ip 172.18.0.4 -container-port 64294
    4680 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 64297 -container-ip 172.18.0.4 -container-port 64297
    4687 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 64297 -container-ip 172.18.0.4 -container-port 64297
    4725 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 2575 -container-ip 192.168.96.2 -container-port 2575
    4733 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 2575 -container-ip 192.168.96.2 -container-port 2575
    4776 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 443 -container-ip 172.27.0.2 -container-port 443
    4787 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 443 -container-ip 172.27.0.2 -container-port 443
    4802 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 2087 -container-ip 172.27.0.2 -container-port 2087
    4808 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 2087 -container-ip 172.27.0.2 -container-port 2087
    4865 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 1025 -container-ip 172.26.0.2 -container-port 1025
    4873 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 1025 -container-ip 172.26.0.2 -container-port 1025
    4927 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 50100 -container-ip 172.26.0.2 -container-port 50100
```

systemctl status docker showing docker-proxy forwarding host ports (3389, 8080, 5060, 25, 443, and others) into individual honeypot containers.

T-Pot's installer automatically relocates SSH off port 22 as part of hardening, since the honeypot deliberately occupies common service ports as bait. That's obscurity, not a control: it cuts noise from opportunistic scanners but doesn't replace key-based auth. The habit that actually matters is verifying the SSH host key fingerprint on first connect, since an unexpected change on a box you've connected to before is one of the few reliable signs of a man-in-the-middle condition.

Running the full T-Pot suite and Splunk together on the original 8GB instance caused memory contention and an unresponsive host. The fix was two-part: disable T-Pot's now-redundant internal Elasticsearch/Kibana/Logstash stack once Splunk took over that role, and resize to 16GB RAM / 8 vCPU. Kibana's own dashboard was still useful early on for a quick gut-check of attack volume before Splunk ingestion was fully wired up.



Kibana's built-in T-Pot dashboard, captured early: 97 attacks in the prior 24 hours across multiple honeypots and source countries (US, Seychelles, Portugal, South Korea, China).

Splunk SIEM Integration

What I Did

Splunk Enterprise was installed directly on the VPS. Its default web UI port (8000) collided with Honeytrap, which binds broadly across common ports by design. The first install attempt failed outright when Splunk detected the port was already bound.

```
Splunk> Be an IT superhero. Go home early.

Checking prerequisites...
  Checking http port [8000]: not available
ERROR: http port [8000] - port is already bound. Splunk needs to use this port.
Would you like to change ports? [y/n]: n
Exiting...

root@honeypot-project-s:/home/tpotuser/tpotce# sudo lsof -i :8000
sudo: unable to resolve host honeypot-project-s: Name or service not known
COMMAND  PID USER  FD   TYPE DEVICE SIZE/OFF NODE NAME
honeypot 73529 tpot   12u  IPv4 832128   0t0  TCP *:8000 (LISTEN)

root@honeypot-project-s:/home/tpotuser/tpotce# sudo /opt/splunk/bin/splunk start --answer-yes
sudo: unable to resolve host honeypot-project-s: Name or service not known

Splunk> Be an IT superhero. Go home early.

Checking prerequisites...
  Checking http port [8000]: not available
ERROR: http port [8000] - port is already bound. Splunk needs to use this port.
Would you like to change ports? [y/n]: y
Enter a new http port: 8001
```

Diagnosing the port 8000 conflict with lsof (Honeytrap already listening) before relocating Splunk's web UI.

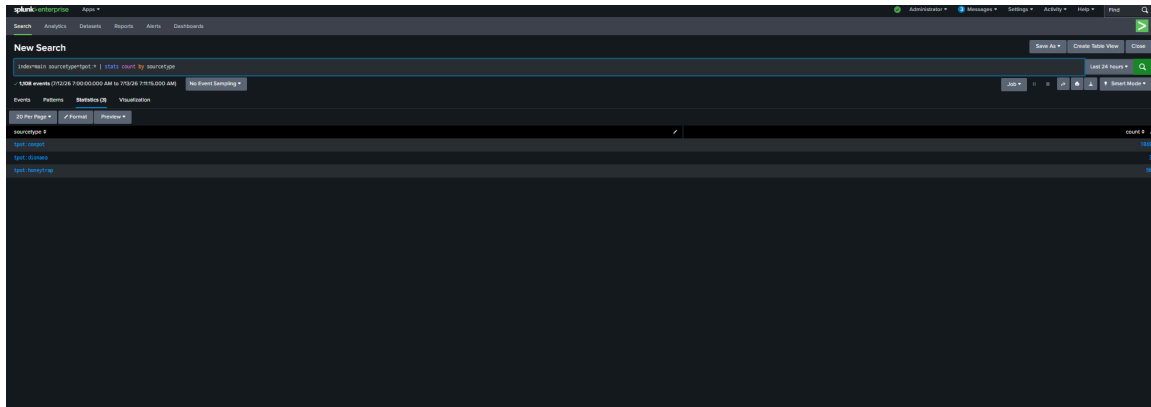
Rather than fighting Honeytrap's port range, Splunk's web UI was relocated, eventually settling on port 8890 after further conflicts. Five honeypot log sources were then ingested into Splunk's main index via the splunk add monitor CLI, each tagged with its own sourcetype so they could be queried independently or together:

Sourcetype	Source path
tpot:cowrie	.../data/cowrie/log/cowrie.json
tpot:honeytrap	.../data/honeytrap/log/attackers.json
tpot:dionaea	.../data/dionaea/log/dionaea.json
tpot:suricata	.../data/suricata/log/eve.json
tpot:conpot	.../data/conpot/log

The commands were run sequentially rather than in parallel; running them concurrently caused config file conflicts in Splunk's monitor configuration.

Verifying Data Flow

A stats query broken down by sourcetype confirmed the ingestion pipeline scaling as traffic accumulated, over 1,100 events indexed across Conpot, Dionaea, and Honeytrap alone, with Conpot's ICS-protocol scanning traffic dominating the early volume.



index=main sourcetype=tpot: | stats count by sourcetype, showing 1,108 total events with tpot:conpot accounting for the majority.*

The traffic mix told a fairly typical story for an unadvertised, internet-facing IP: most hits were automated, opportunistic scanning rather than targeted intrusion. Source IPs spanned multiple continents within the first day alone; a honeypot doesn't need to be advertised or linked anywhere to attract traffic, internet-wide scanning by both botnets and research crawlers finds any exposed IP within hours.

Deception Layer: Honeytokens

What I Did

Two honeytokens were deployed with Canarytokens.org to add a high-fidelity signal that fires only on genuine interaction with planted bait, unlike the honeypots themselves, which log volume from any automated scanner:

- Web bug token: embedded in a fake backup_credentials.txt file, referencing a fake internal admin panel URL. Fetching that URL fires an alert.
- AWS credential token: a fake AWS access key and secret placed at the standard ~/.aws/credentials path, exactly where an attacker or post-exploitation tool checks first on a compromised Linux host.

```
root@honeypot-project-s:/home/tpotuser# cat /home/tpotuser/backup_credentials.txt && echo "---" && cat /home/tpotuser/.aws/credentials
# Internal Backup Server Credentials
# DO NOT SHARE - Last updated: 2026-06-15

Backup Server: backup-internal.corp.local
Admin Panel: http://canarytokens.com/articles/tags/rbgzs08wl0ynfj1m71oe3jhls/post.jsp
Username: svc_backup_admin
Password: [REDACTED - see password manager]

Notes: Rotate credentials quarterly per security policy.

---
[default]
aws_access_key_id = AKIAUE4EELJIQRWUISOD
aws_secret_access_key = yhBemTymdWeAmIKQ8TxxRYg99FRAClaIcMojJUYq
output = json
region = us-east-2
root@honeypot-project-s:/home/tpotuser#
```

Terminal output confirming both honeytoken files are in place: the fake backup_credentials.txt and the fake ~/.aws/credentials file.

Both tokens were placed to mimic real attacker tradecraft rather than random file paths, which is the actual point of a honeytoken: the bait has to look like something a real intrusion would produce or rely on. Either token firing triggers an email alert independent of the Splunk pipeline, so the deception layer still works as a tripwire even if the SIEM ingestion path is down.

LLM-Powered Threat Classification

What I Did

The classification pipeline is a Python script (`honeypot_llm_classifier.py`) that pulls recent attack log entries from Splunk over its REST API, sends each event to Claude with a prompt requesting attack categorization, MITRE ATT&CK mapping, and a severity rating in strict JSON, then writes the enriched result back into Splunk over HTTP Event Collector under sourcetype `tpot:llm_enriched`. A `--mode` flag (test or live) and a `--write` flag (dry run or actual HEC write) made it safe to validate the logic before it touched the index.

Test mode was run first against five sample log entries. The model correctly distinguished brute-force activity, malware delivery, exploitation attempts, and reconnaissance scanning, mapping each to a specific MITRE ATT&CK technique ID.

```
python3 /home/tpotuser/honeypot_llm_classifier.py --mode test
Running in TEST mode against 5 sample log entries (limit=5).

[1/5] Classifying event from 185.220.101.45 against honeypot 'cowrie'...
-> brute_force (T1110.001: Brute Force: Password Guessing) severity=medium
(dry run, not writing to Splunk HEC)

[2/5] Classifying event from 185.220.101.45 against honeypot 'cowrie'...
-> malware_delivery (T1105: Ingress Tool Transfer) severity=critical
(dry run, not writing to Splunk HEC)

[3/5] Classifying event from 45.155.204.12 against honeypot 'dionaea'...
-> exploit_attempt (T1210: Exploitation of Remote Services) severity=critical
(dry run, not writing to Splunk HEC)

[4/5] Classifying event from 91.240.118.33 against honeypot 'wordpot'...
-> brute_force (T1110: Brute Force) severity=high
(dry run, not writing to Splunk HEC)

[5/5] Classifying event from 194.26.29.14 against honeypot 'conpot'...
-> recon_scan (T0846: Remote System Discovery) severity=high
(dry run, not writing to Splunk HEC)

root@honeypot-project-s:/home/tpotuser#
```

Classifier in --mode test against five sample events: brute force (T1110.001), ingress tool transfer (T1105), exploitation of remote services (T1210), and remote system discovery (T0846).

With the logic validated, the script was run in `--mode live` against real Splunk events. It correctly flagged a run of reconnaissance and service-discovery activity as lower severity relative to the exploitation attempts seen in the test set, consistent with what the underlying honeypot logs showed for that window.

```
python3 /home/tpotuser/honeypot_llm_classifier.py --mode live
/usr/lib/python3/dist-packages/urllib3/connectionpool.py:1032: InsecureRequestWarning: Unverified HTTPS request is being made to host 'localhost'. Adding ce
rtificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/latest/advanced-usage.html#ssl-warnings
warnings.warn(
Pulled 5 live log entries from Splunk (limit=5).

[1/5] Classifying event from unknown IP against honeypot 'unknown'...
-> brute_force (T1110.001: Brute Force: Password Guessing) severity=medium
(dry run, not writing to Splunk HEC)

[2/5] Classifying event from unknown IP against honeypot 'unknown'...
-> recon_scan (T1046: Network Service Discovery) severity=low
(dry run, not writing to Splunk HEC)

[3/5] Classifying event from unknown IP against honeypot 'unknown'...
-> recon_scan (T1046: Network Service Scanning) severity=low
(dry run, not writing to Splunk HEC)

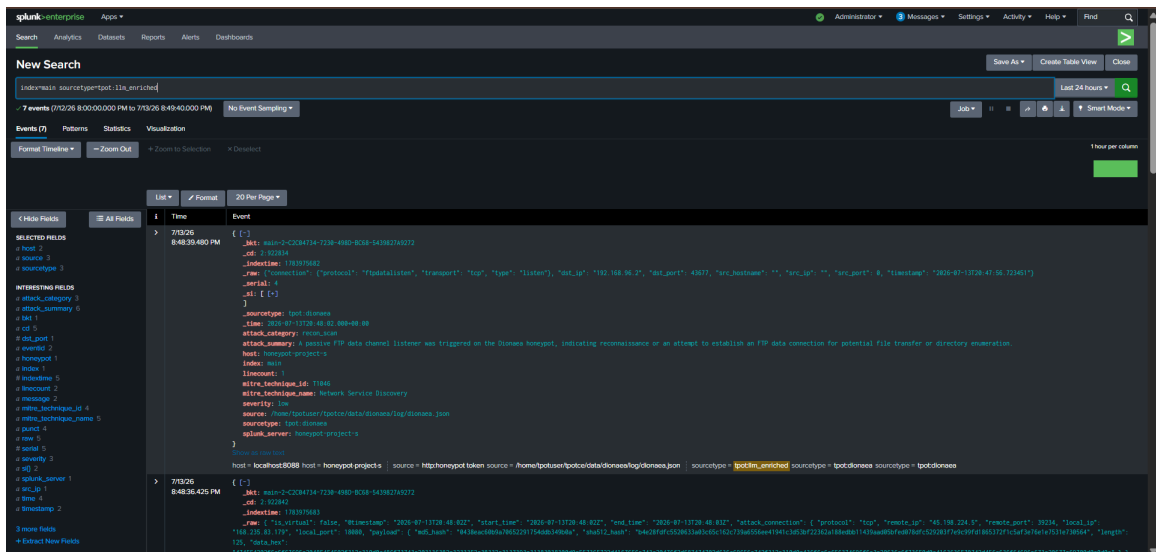
[4/5] Classifying event from unknown IP against honeypot 'unknown'...
-> recon_scan (T1595: Active Scanning) severity=low
(dry run, not writing to Splunk HEC)

[5/5] Classifying event from unknown IP against honeypot 'unknown'...
-> recon_scan (T1046: Network Service Discovery) severity=low
(dry run, not writing to Splunk HEC)

root@honeypot-project-s:/home/tpotuser#
```

Classifier in --mode live, pulling five real events from Splunk and classifying them as brute force and reconnaissance activity (T1110.001, T1046, T1595).

Once validated, the script was run with `--write` enabled to close the loop. Querying sourcetype=`tpot:llm_enriched` showed fully enriched events, `attack_category`, `attack_summary`, `mitre_technique_id`, `mitre_technique_name`, and `severity`, populated alongside the original raw honeypot fields.



A fully enriched event under `sourcetype=tpot:llm_enriched`: original Dionaea log fields alongside the LLM-generated `attack_category`, `mitre_technique_id`, `mitre_technique_name`, and `severity` fields.

Notable Troubleshooting

RamNode Network Reachability

Shortly after provisioning, the instance was completely unreachable externally (SSH, web UI, even ICMP) despite correct server-side configuration. MTR was run from multiple independent networks to rule out a local ISP issue, and a second RamNode instance in the same region served as a control to confirm the problem was isolated to the one IP. With that evidence, a support ticket was opened; RamNode confirmed and resolved a network-side routing issue on their end.

LLM API Selection

The original plan was Google's Gemini API to keep the project fully free. API keys generated through Google's console returned OAuth-style tokens prefixed AQ instead of standard API keys, causing 401 ACCESS_TOKEN_TYPE_UNSUPPORTED and 429 quota errors across both auth methods. The issue reproduced across two separate Google accounts, ruling out an account-specific misconfiguration and pointing to a platform-side bug. The project switched to the Anthropic API, which worked on the first attempt.

MITRE ATT&CK Mapping

Mapping the classifier's output to MITRE ATT&CK gives a security lead a framework-level view of what the honeypot actually observed:

Tactic	Technique	ID	Telemetry Source
Credential Access	Brute Force: Password Guessing	T1110.001	Cowrie (SSH/Telnet)
Command and Control	Ingress Tool Transfer	T1105	Cowrie / Dionaea
Initial Access	Exploitation of Remote Services	T1210	Dionaea
Discovery	Remote System Discovery	T1046	Conpot / Honeytrap
Reconnaissance	Active Scanning	T1595	Suricata / Honeytrap

Key Metrics Summary

Metric	Value
VPS specs	16GB RAM / 8 vCPU / 260GB SSD (RamNode, EWR)
Honeypot daemons deployed	20+ (T-Pot 24.04.1, Hive mode)
Total Splunk-indexed events	1,108+ (index=main, sourcetype=tpot:*)
Distinct source countries (24h)	6+ (US, Seychelles, Portugal, South Korea, China, and others)
Honeytokens deployed	2 (web bug + fake AWS credentials)
LLM-enriched events written back	Validated end to end in live mode

Lessons Learned

1. **Co-locating Splunk and T-Pot on one VPS was the right tradeoff for this scope.** It removed a networking problem at the cost of a resource-management problem, which was easier to solve, and disabling the now-redundant internal ELK stack early would have avoided the initial memory pressure entirely.
2. **Sequential execution matters for CLI-driven configuration changes.** Running the splunk add monitor commands in parallel produced config file conflicts that running them one at a time avoided.
3. **Reproduce a vendor API failure across accounts before calling it platform-side.** The Gemini token issue held across two separate Google accounts, which is what made it a defensible root cause instead of a guess.
4. **A honeypot doesn't need to be advertised to get discovered.** Internet-wide scanning by both automated botnets and research crawlers found this instance within hours of it going live.
5. **A deception signal is only as good as its independence from the primary pipeline.** Honeytoken alerts fire by email regardless of Splunk's state, so the tripwire still works even if the SIEM ingestion path goes down.

Future Work

- Run the LLM classifier on a schedule (cron or a Splunk scheduled search) instead of invoking it manually, so enrichment happens continuously.
- Add alerting on top of the tpot:llm_enriched sourcetype for anything classified as critical severity.
- Expand the honeytoken set (a fake SSH private key, a fake .env file) to widen the deception surface.

Tools & Technology Stack

- Honeypot suite: T-Pot 24.04.1 (Hive mode), Docker-based containers
- SIEM: Splunk Enterprise (self-hosted on the VPS)
- Cloud provider: RamNode KVM VPS (Ubuntu 22.04, EWR region)
- Deception: Canarytokens (web bug + fake AWS credentials)
- LLM integration: Anthropic API (Claude), via a custom Python classification script
- Networking diagnostics: MTR